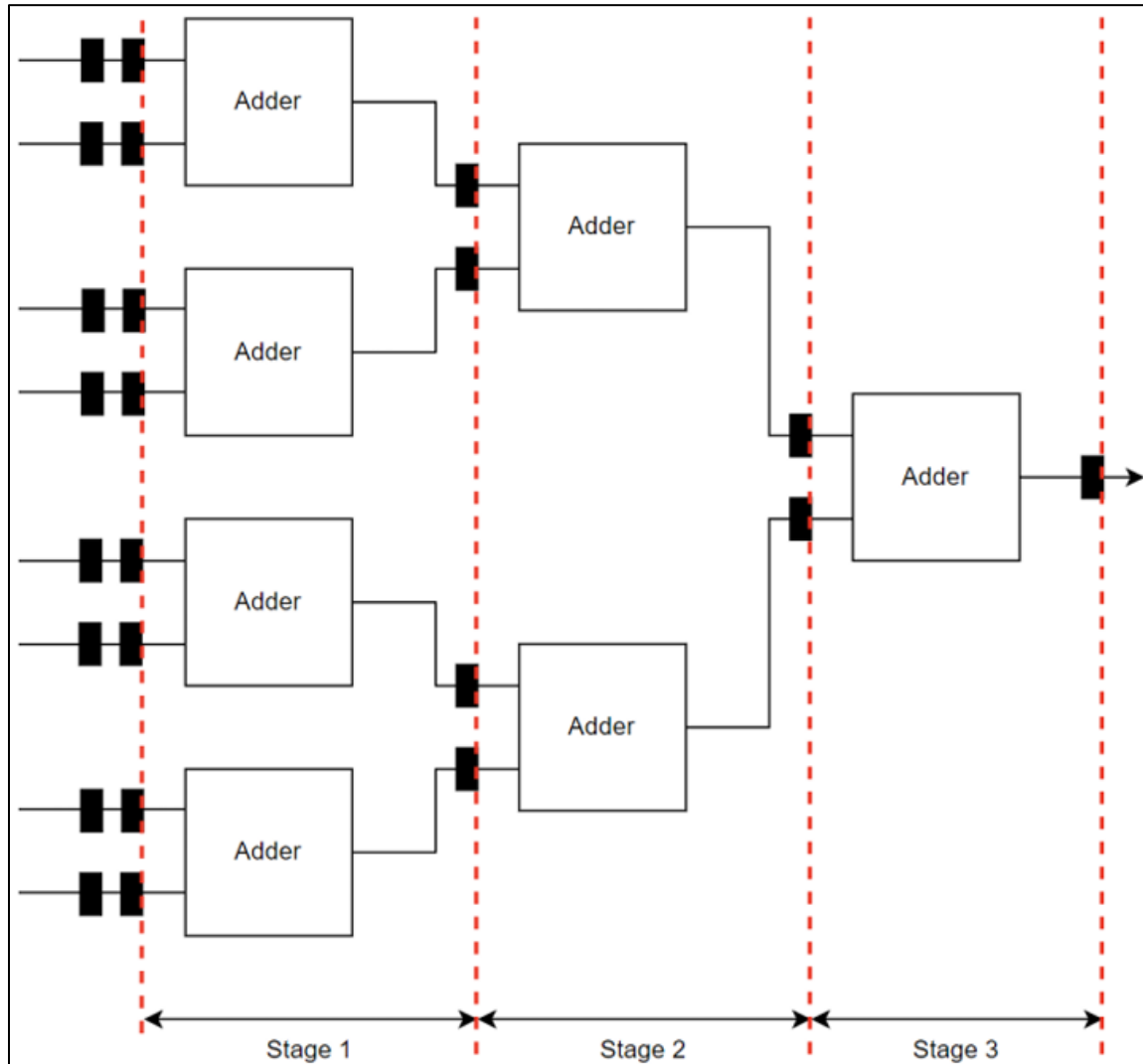


LAB 7: Adder Tree

Date: 02 June of 2025



Proffesor: Dr. Francisco Javier Rodríguez Navarrete

Students: Cesar Eduardo Inda Cenicerros

Alonso Emmanuel López Macías

Team: 14

Introduction

In digital circuit design, adders are fundamental building blocks used in a wide range of applications, from arithmetic logic units (**ALUs**) to **digital signal processors (DSPs)**. An advanced and efficient structure for performing multiple additions simultaneously is the adder tree, which improves speed and performance in parallel operations, such as data accumulation from sensors, digital filters, or large-scale processing algorithms. This lab aims to implement a **parameterizable** adder tree in Verilog, capable of performing 8 additions in parallel, with the design segmented by intermediate registers connecting each adder's output to the next level in the structure. The input word size is parameterizable, allowing easy adaptation of the design to different applications and data width requirements.

The adder tree is widely used in digital electronics, particularly in parallel processing architectures such as **FPGAs**, **ASICs**, and specialized processors. Understanding its structure, functionality, and implementation in HDL like Verilog is essential for developing efficient solutions in projects that demand speed and resource optimization. Additionally, as an extra challenge, a more generalized and fully parameterizable version of the adder tree is proposed, capable of operating with **N inputs, where N is a multiple of 2**. This also requires adapting the testbench, thus offering greater flexibility and scalability in the design.

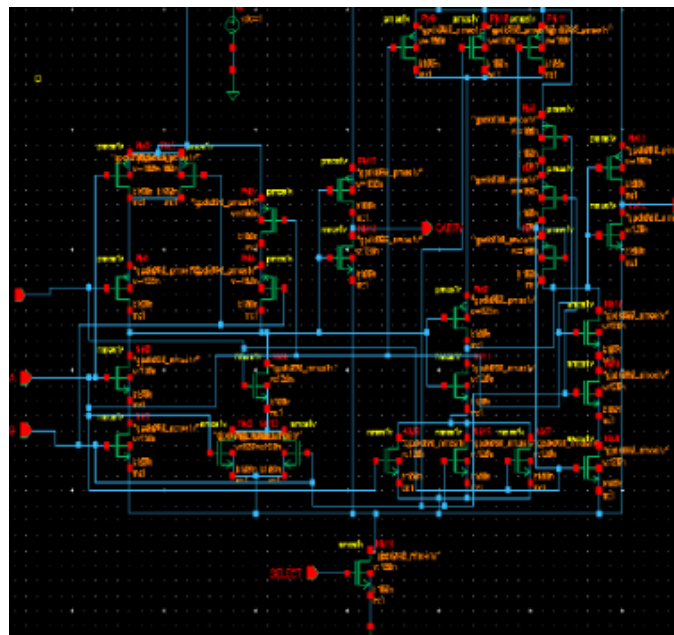


Figure: 1 FullAdder_Cmos-Diagram

Requirements

Design an adder tree with the following characteristics:

- *The word size of the adders must be parameterizable.*
- *The adder tree must be capable of performing 8 additions in parallel.*
- *The adder tree must be segmented such that the output of each adder is connected to a register.*

To verify the functionality of your design, use the following testbench code. Additionally, for extra points, you shall submit a parameterized version of the adder tree, that can be configured for parameter N that is a multiple of 2. This would require changing the testbench too.

Development

For all considerations was better use a **Visual Studio Code Extension** for see easier the sintaxis in the highlight's words, because Quartus use a black text in so much context and this is some difficulty for write a big text or descriptions. In this we use this extension:

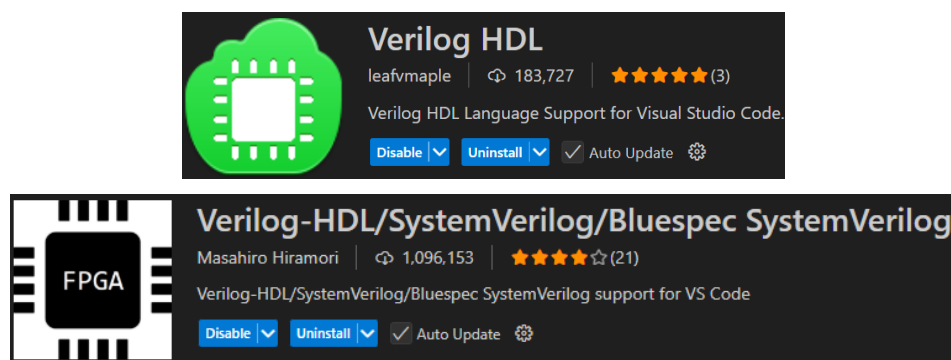


Figure: 2 Verilog Visual Studio Extension

Testbench

In this laboratory, the design was not loaded onto the FPGA; it was only tested **using waveforms in ModelSim-Altera** through the testbench. This approach allows for handling a parameterizable adder without encountering issues related to the number of inputs that would be required on the board.

In the lab was possible see the **“testbench”** section.

```
*timescale 1ns / 1ps

module adder_tree_tb;

    parameter DATA_WIDTH = 32;
    parameter TREE_INPUTS = 8;

    bit clk;

    logic [DATA_WIDTH-1:0] data_a [TREE_INPUTS];
    logic [DATA_WIDTH-1:0] data_b [TREE_INPUTS];
    logic [DATA_WIDTH-1:0] data_out;

    localparam STAGES = $clog2(TREE_INPUTS) + 1;

    // DUT
    adder_tree #(
        .DATA_WIDTH(DATA_WIDTH),
        .TREE_INPUTS(TREE_INPUTS)
    ) dut (
        .clk(clk),
        .data_a(data_a),
        .data_b(data_b),
        .data_out(data_out)
    );
```

Module Declaration & DUT → TestBench

```
// Clock
always #5 clk = ~clk;

//TOFIXME declare logics 32bits
logic [31:0] result;
logic [31:0] val_a, val_b;
integer n;

/*integer result;
integer val_a, val_b;
integer n;*/

initial begin
    clk = 0;

    repeat(50) begin
        @(negedge clk);
        result = 0;

        //Aqui se generan los datos aleatorios
        for (n = 0; n < TREE_INPUTS; n = n + 1) begin
            val_a = $random;
            val_b = $random;
            data_a[n] = val_a;
            data_b[n] = val_b;
            result = result + val_a + val_b;
        end
```

Cycle (50) & RandomValues → TestBench

```
//Aqui espera la propagacion completa (STAGES + 2 ciclos)
repeat (STAGES + 2) @(negedge clk);

//TOFIXME change to "unsigned"
// Mostrar y verificar
$display("Iteration %0t ns -> data_out = %0d | expected = %0d", $time, data_out, result);
if (data_out != result) begin
    $display("ERROR: Result is incorrect.");
    $stop;
end
end

//show this if the iterations were correct.
$display("All the iterations were passed.");
$finish;
end

endmodule
```

Propagation & Display → TestBench

Figure: 3 AdderTree_Testbench.sv

The implementation with the other part of the logic was as following:

```
module adder_tree #(
    parameter DATA_WIDTH = 32,
    parameter TREE_INPUTS = 8 // Debe ser una potencia de 2: 2, 4, 8, 16, ...
)()
    input logic clk,
    input logic [DATA_WIDTH-1:0] data_a [TREE_INPUTS],
    input logic [DATA_WIDTH-1:0] data_b [TREE_INPUTS],
    output logic [DATA_WIDTH-1:0] data_out

);

    // Primer paso: sumar A + B
    logic [DATA_WIDTH-1:0] stage0 [TREE_INPUTS];

    genvar i;
    generate
        for (i = 0; i < TREE_INPUTS; i = i + 1) begin : gen_sum_ab
            always_ff @(posedge clk)
                stage0[i] <= data_a[i] + data_b[i];
        end
    endgenerate
```

Parametrization, SUM & Variables Declaration → AdderTree

```
genvar i;
generate
    for (i = 0; i < TREE_INPUTS; i = i + 1) begin : gen_sum_ab
        always_ff @(posedge clk)
            stage0[i] <= data_a[i] + data_b[i];
    end
endgenerate

// Definir cuantas etapas se requieren
localparam STAGES = $clog2(TREE_INPUTS);

// Definir todas las etapas intermedias
logic [DATA_WIDTH-1:0] stage [0:STAGES][TREE_INPUTS-1:0];

// Inicializar stage[0] con stage0
generate
    for (i = 0; i < TREE_INPUTS; i = i + 1) begin : gen_stage0_copy
        always_ff @(posedge clk)
            stage[0][i] <= stage0[i];
    end
endgenerate
```

Bidirectional Matriz & Array Transfer Value → AdderTree

```
genvar s, j;
generate
    for (s = 1; s <= STAGES; s = s + 1) begin : stage_loop
        for (j = 0; j < (TREE_INPUTS >> s); j = j + 1) begin : sum_loop
            always_ff @(posedge clk)
                stage[s][j] <= stage[s-1][2*j] + stage[s-1][2*j+1];
        end
    end
endgenerate

// Salida final del arbol
always_ff @(posedge clk)
    data_out <= stage[STAGES][0];

endmodule
```

Pair of Sums & DataOut → AdderTree

Figure: 4 AdderTree.sv

The result in the testbench is as following for see in the **monitor and in the waveforms**.

Is only a brief for waveforms because are 50 cycles or iterations

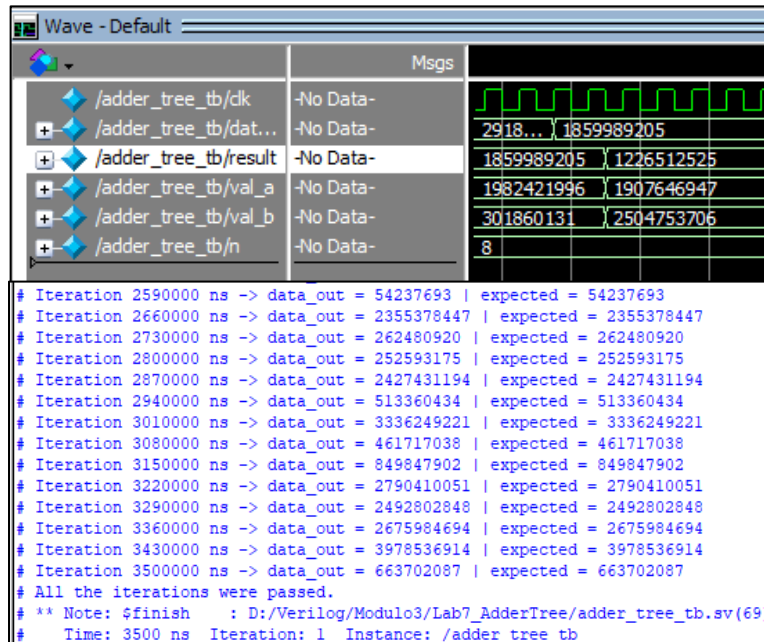


Figure: 5 AdderTree - ModelSim Display Waveform

FPGA Implementation

Compilation verification:

Flow Status	Successful - Wed Jun 04 16:38:20 2025
Quartus Prime Version	18.1.0 Build 625 09/12/2018 SJ Lite Edition
Revision Name	adder_tree
Top-level Entity Name	adder_tree
Family	Cyclone IV E
Device	EP4CE115F29C7
Timing Models	Final
Total logic elements	768
Total registers	768
Total pins	545
Total virtual pins	0
Total memory bits	0
Embedded Multiplier 9-bit elements	0
Total PLLs	0

Figure: 6 Compilation Report

In this case aren't pin planner because only was runned in testbench, for the requeriments in the physical board. But this the representation of the **RTL**.

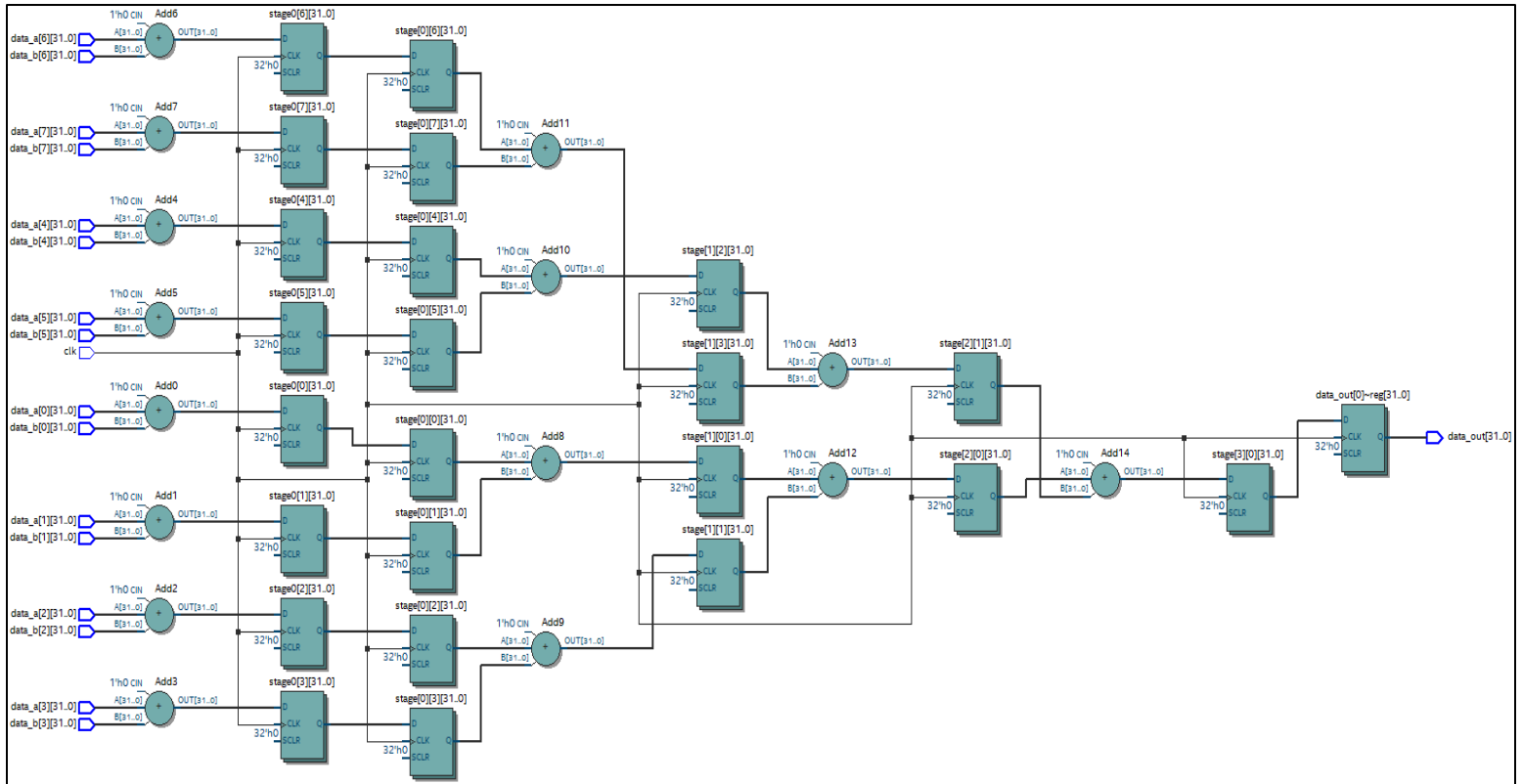


Figure: 7 RTL Viewer

GitHub Repository

https://github.com/CeicGitHub/Fundamentals_Of_Digital_Design_FPGA_3/tree/Lab7_AdderTree

Conclusion / Issues

Cesar Eduardo Inda Cenicerros

In this lab, the activity was somewhat different from previous practices, as it consisted solely of developing the testbench. This was because fully synthesizing the program requires multiple input signals, and therefore, simulation was the only feasible option in this case. We focused exclusively on that part and implemented it in System Verilog.

One of the first complications we faced was the need to fully understand the base logic, since making the adder tree parameterizable required a solid foundational understanding to avoid falling into common mistakes. Later, while generating the testbench, we encountered issues because the variable type **"logic"** had not been defined properly—initially, it was only declared as an **"integer"**. As a result, the output began displaying signed representations instead of the unsigned or logical output we expected to see.

Alonso Emmanuel Lopez Macias

One of the main sections of the lab where we began to face more difficulties was understanding how the stage transitions would occur for the array values, we had previously defined. This became a significant issue because, in order to implement it properly, we had to consider many details, starting with generating random numbers, then accounting for signal propagation, and identifying the exact moment when the clock would capture the value.

After that, in our main logic, we had to work on the iteration and partial sum sections. In other words, the binary or adder tree had to take many of these considerations into account to produce the correct result. This led us to repeatedly test different values in the Altera simulator, iterating and adjusting each one. Additionally, having a solid base in the logic was extremely helpful. Another key aspect was working with the parameterizable part of the adder specifically, handling the required 32 bits and adjusting the parameter as a power of 2 for the corresponding modification.